Harbour House

Hotel Reservations Database Management System

COMP9120 Group Project Report | Group 60

Team members and SIDs: [Zirui Meng,550593111],[JinXing Wu,560555079],[Tianyi Shen,560025675]

Project Overview

This project is to design a hotel reservation and room-management system. The system manages single-hotel operations such as booking a room and checking-in. It should be able to solve room booking conflicts, track guest and room status, and stay extensions or transfers under strict date and concurrency constraints.

This system is for mainly 2 kinds of users: guest and hotel staff. It should allow guests to perform these actions: search room availability, create/cancel bookings, view stay history, and approve stay extension quotes. Hotel staff can also use this system to register occupants, execute check-ins/outs, update housekeeping states, and process room transfers.

A 7-table PostgreSQL relational database ensuring strict transactional integrity:
- Guest: Primary contact details.
- RoomType: Category rates and max capacity.
- Room: Room numbers, bookability flag, and housekeeping status.
- Reservation: Dates, locked rates, status lifecycle, actual timestamps, and extension links.
- ReservationGuest: Roster of actual staying occupants per reservation.
- Staff: Authorised hotel manager or receptionist
- Reservation_aduit: Recording changes of any reservation.

This project uses these 3 core functions to ensure this system works correctly:

- Reservation Lifecycle: Prevents overlapping bookings, locks rates upon creation, and records exact actual check-in/out timestamps.

- Housekeeping Management: Decouples bookability from room cleanliness (clean, dirty, cleaning), setting rooms to 'dirty' upon checkout.

- Extensions & Transfers: Handles continuous stay extensions using self-referencing links (`previous_reservation_id`) and executes room swaps within single atomic transactions.

1. Problem and Requirements

1.1 Problem and Motivation

Managing hotel operations requires coordinating room availability, pricing, cleaning states, and guest occupants without scheduling conflicts. A centralized relational database system ensures transaction safety, prevents double-booking conflicts, decouples future bookability from real-time housekeeping, and tracks stay extensions or room swaps via chained reservation segments. For guests, this provides transparent booking and quote confirmation. For staff, it eliminates double-booking risks and streamlines check-ins, check-outs, and room maintenance.

1.2 Users and Stakeholders

-Guests (Booking Contacts): Unverified individuals who search for rooms, make initial bookings, view/cancel their reservations, and confirm extension quotes. They do not hold authenticated customer accounts.

- Actual Occupants (Staying Guests): Individuals registered on a reservation's occupant roster upon check-in. The contact guest may or may not be an occupant.

- Hotel Staff (Receptionists & Managers): Authenticated internal users who manage occupant rosters, process check-ins and check-outs, update housekeeping statuses, handle stay extensions or room changes, and apply manual discounts or overrides.

1.3 Data Requirements

The database should be able to store data with types below:
-Guest: Guest id(PK), Name, phone number, and email.
-Room Types: type id(PK), type name, maximum guest capacity, and base nightly rate.
-Rooms: room id(PK), room number, room type, bookability flag, and housekeeping status (clean, dirty, cleaning).
- Reservation: Fixed booked nightly rate, planned check-in/check-out dates, guest count, status (confirmed, checked_in, checked_out, cancelled), actual check-in/out timestamps, and a self-referencing link (`previous_reservation_id`) for extensions.
-Reservation_Guest(Occupants): Roster entries mapping individual occupant names to a specific reservation.
-Reservation_aduit(Operations): Transaction logs tracking staff actions, state transitions, manual discounts, and operational history.

1.4 Business Rules and Scope

1. A reservation corresponds to exactly one specific room and one guest.

2. The guest represents the booking contact only. Actual occupants are recorded separately in Reservation_Guest; if the booking contact also stays, they must be registered explicitly as well.

3. Every room must belong to a single room type, and room numbers are globally unique across the hotel.

4. The stay period has a minimum of one night, operating under the Australia/Sydney timezone. The check-in date cannot be earlier than the hotel's current date.

5. Guest_count must be a positive integer <= max_guests of the RoomType. At check-in, registered occupant count must match guest_count.

6. Active bookings (confirmed, checked_in) on the same room cannot overlap in dates. Cancelled and checked_out bookings are ignored.

7. Overlap occurs if (existing.check_in < new.check_out AND existing.check_out > new.check_in). Adjacent dates do not overlap.

8. Search checks availability without holding stock. Creation re-checks availability inside a transaction and locks the rate.

9. Each reservation locks a fixed booked_nightly_rate. Base rate updates do not affect existing bookings. Total amount = planned nights * booked rate.

10. Valid status flows: confirmed -> checked_in / cancelled; checked_in -> checked_out. Terminal states cannot be reopened.

11. Check-in allowed on or after planned check-in date before check-out date. Late arrivals do not automatically adjust dates/prices.

12. The room must be bookable (is_bookable) and clean (housekeeping_status = 'clean') with a full occupant roster and no active stay.

13. Only checked_in reservations can check out. Records actual timestamps without automatic refund or overstay penalty calculations.

14. is_bookable manages future availability. Housekeeping_status tracks 'clean', 'dirty', or 'cleaning'. Pending cleaning does not block future bookings.

15. Checking out and setting room to 'dirty' commit in the same transaction. Flow: dirty -> cleaning -> clean (confirmed by staff).

16. Occupied rooms cannot be set to dirty/cleaning. Closing a room does not auto-cancel existing bookings.

17. Extensions apply only to active (checked_in) stays. Check the current room first, then the same room type, then alternative types.

18. Cannot overlap existing valid bookings. Upgrades are not free by default and require explicit guest confirmation.

19.Creates a new continuation Reservation segment linked via previous_reservation_id. Each segment retains its own rate.

20. Room swap finishes the old reservation, sets old room to 'dirty', and starts a new reservation in a single transaction. The new room must be clean.

21. Same-room continuation finishes the old reservation and starts a new segment in one transaction WITHOUT setting the room to 'dirty'.

22. If no room is available or guest declines, original booking remains unchanged. Mid-stay transfers or auto-extensions are unsupported.

23. Self-references and loops are prohibited. Uncancelled successors must be unique per segment. Contact guests must remain consistent.

1.5 Representative User Operations

1. Guests query available room types for a date range and guest count, locking in a booking segment at current base rates.

2. Staff register actual staying occupants prior to check-in, execute check-in on clean rooms, and process check-outs to mark rooms dirty.

3. Staff extend an active stay by creating a linked continuation segment (same room or room swap) upon guest approval.

4. Staff or guests adjust planned dates, room selections, or contact details before the check-in date.

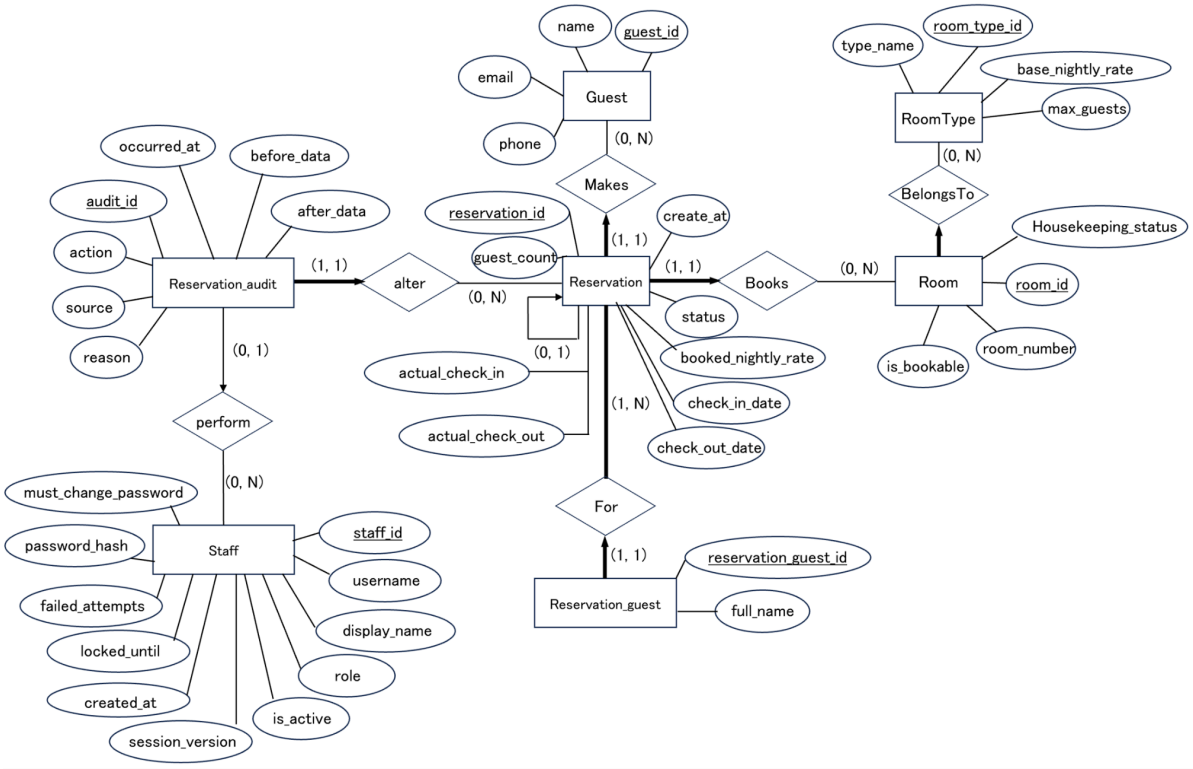
5. Staff simultaneously create atomic reservations for multiple rooms across a single transaction.

6. Authorized managers apply manual rate discounts or price overrides during reservation creation or extension quoting.

7. Staff review historic reservation events, housekeeping state transitions, and system audit logs.

2. Conceptual Design (ER Model)

2.1 Final ER Diagram



2.2 Entities and Attributes

There are 7 entities in the final ER diagram: Guest, Room, RoomType, Reservation, Reservation_guest, Reservation_audit, Staff.

Guest: the booking contact, it has these attributes: guest_id(PK), name, email, phone.

Reservation: represents a booking made by a guest for a stay. Includes a self-referencing relationship to track room change. Its primary key is reservation_id.

Room_Type: categorizes rooms into specific types with shared pricing and capacity limits.

Room: represents individual physical rooms inside the hotel.

Reservation: represents additional or specific guest details tied to a reservation.

Staff: represents employees or system users who manage operations, log actions, and handle reservations.

Reservation_audit: An audit log tracking changes, updates, or actions performed on reservations over time.

2.3 Relationships and Constraints

Guest Makes Reservation (1-to-Many)
A Guest can make **zero or many** reservations.
A reservation is made by **exactly one** guest.

Reservation Books Room (Many-to-Many)
A Reservation books **one or more** rooms.
A Room can be booked **zero or many** times historically/future-wise.

Room BelongsTo RoomType (Many-to-1)
A RoomType can describe **zero or many** individual rooms.
A Room belongs to **exactly one** room type.

Reservation For Reservation_guest (1-to-Many)
A Reservation can have **one or many** secondary/associated guests.
Each Reservation_guest is tied to **exactly one** reservation.

Reservation alter Reservation (Self-Referencing)
A reservation can modify another reservation to track version history or re-bookings.

Staff perform Reservation_audit (1-to-Many)
A Staff member can perform **zero or many** audit actions.
An audit log entry is performed by **zero or one** staff member.

Reservation_audit alter Reservation (1-to-1 / 1-to-Many)
Links a specific modification audit log back to the altered reservation.

2.4 Design Rationale and Assumptions

The conceptual model translates the core operational requirements of a hotel management system into a robust, scalable entity structure comprising seven distinct entities.

Separation of Guest and Reservation Entities: Decoupling guests from reservations ensures that repeat customers do not duplicate personal details across multiple stays. A one-to-many relationship allows a single guest profile to accumulate a historical record of bookings.

Normalization of Room Specifications: Physical rooms are separated from their category specifications. This prevents data anomalies; attributes like base_nightly_rate and max_guests are managed centrally per room type rather than redundantly across every individual room instance.

Handling Multiple Occupants: While a reservation is spearheaded by a primary guest, hotel stays often involve multiple occupants. The Reservation_guest entity captures secondary guest details (full_name) via a one-to-many relationship, maintaining data integrity without bloating the core Reservation table.

Accountability and Auditing: To enforce security, tracking, and compliance, the Staff entity logs administrative actions. The Reservation_audit entity acts as a historical ledger, capturing state changes (before_data, after_data, action, reason) to maintain accountability over booking modifications.

Self-Referencing Reservation Modifications: The recursive relationship on Reservation handles booking modifications, extensions, or re-bookings cleanly within the same table schema by tracking state transitions.

3. Logical Design and Normalisation

3.1 Relational Schema

- The implemented RM contains seven relations, 48 attributes and seven foreign keys. Figures 1a–1b are drawn and exported in draw.io; the editable source is database/Hotel_Report_RM.drawio.
- PK denotes the primary key; UQ denotes a non-null unique candidate key; FK denotes a foreign key; NN and NULL identify nullability. Core/Staff keys are INTEGER identity values; audit_id is BIGINT identity. Dates are DATE, actual instants are TIMESTAMPTZ and rates are NUMERIC(10,2).

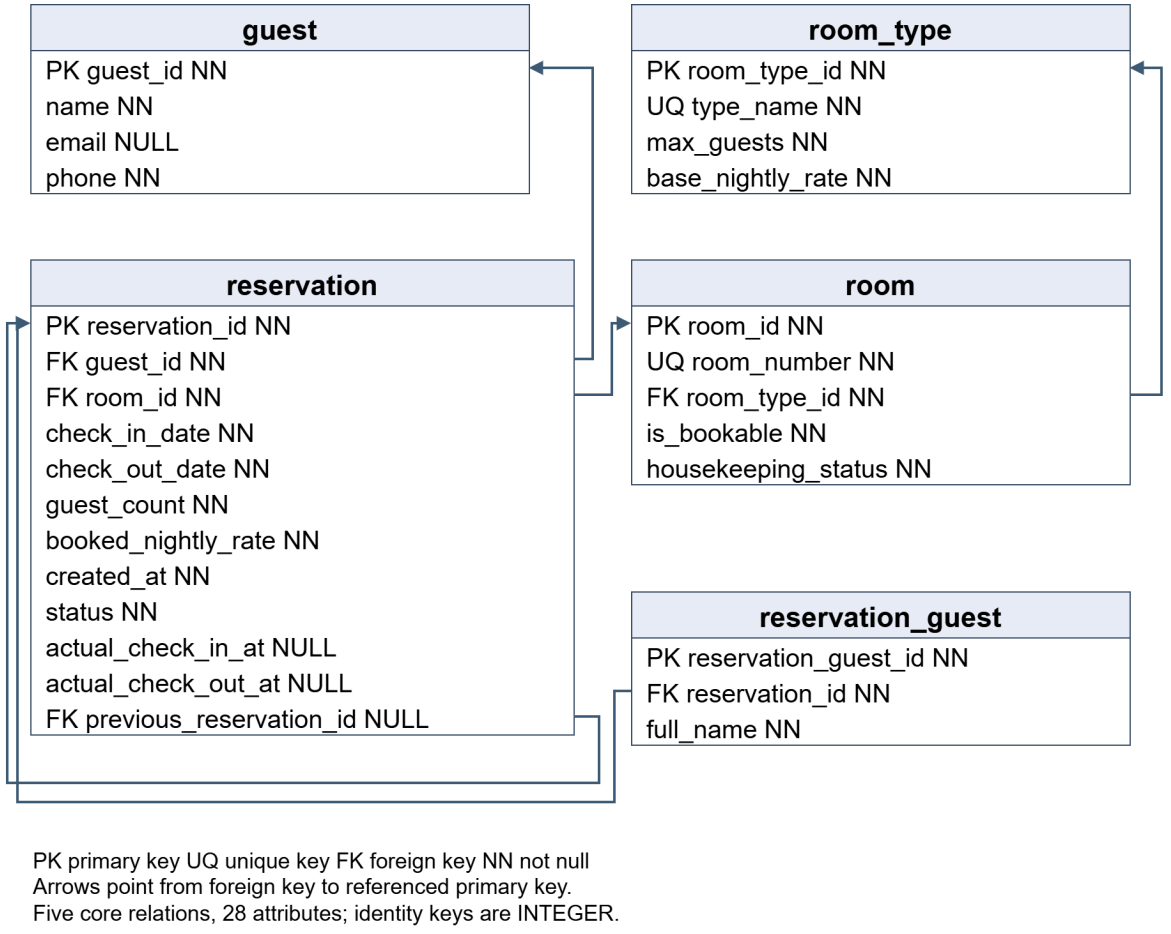


Figure 1a. Core relational model, exported from draw.io: five relations and 28 attributes. Arrows run from FK to the referenced PK.

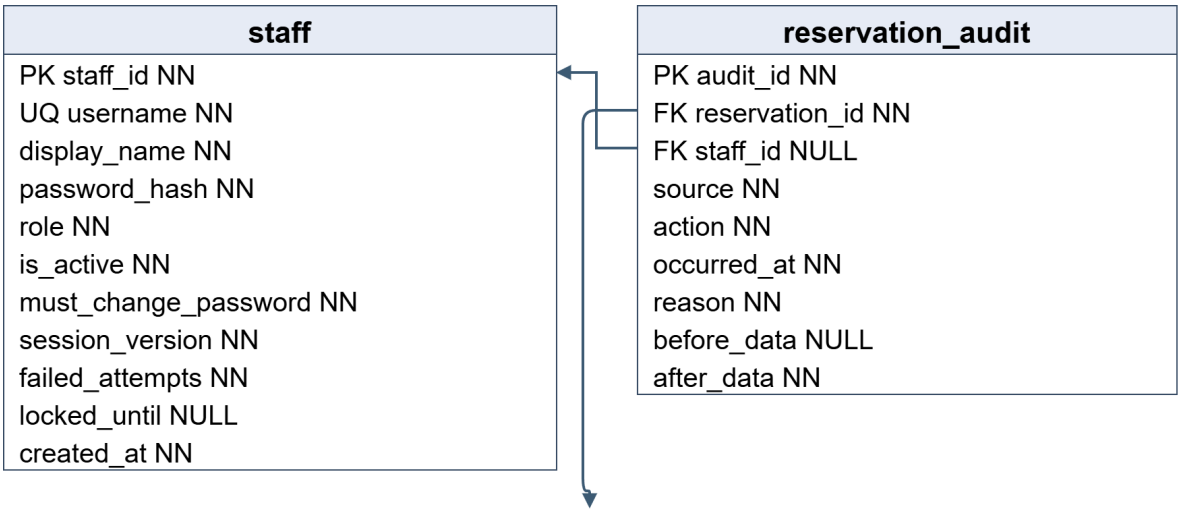


Figure 1b. Staff and ReservationAudit extension, exported from draw.io. Audit.reservation_id references the Reservation PK in Figure 1a; the two panels together show all seven FKs.

Table 1. Candidate keys and foreign-key references.

Relation	Candidate keys	Foreign-key target(s)
Guest	guest_id	None
RoomType	room_type_id; type_name	None
Room	room_id; room_number	room_type_id → RoomType
Reservation	reservation_id	guest_id → Guest; room_id → Room; previous_reservation_id → Reservation (nullable)
ReservationGuest	reservation_guest_id	reservation_id → Reservation
Staff	staff_id; username	None
ReservationAudit	audit_id	reservation_id → Reservation; staff_id → Staff (nullable)

- Names, phone numbers and email addresses are not unique identifiers. Occupants may share a name. The conditional unique indexes in Section 4.3 do not make room_id or previous_reservation_id candidate keys over the whole Reservation relation.

3.2 Mapping from ER Model to Relations

- Entity mapping: Guest, RoomType, Room, Reservation, ReservationGuest, Staff and ReservationAudit each map to a relation with an identity primary key.
- Ordinary 1:N mapping: Room carries room_type_id; Reservation carries guest_id and room_id; ReservationGuest carries reservation_id. Mandatory references use NOT NULL foreign keys; parent rows need not already have children.
- Recursive mapping: previous_reservation_id references Reservation. A root segment has NULL; a continuation has one predecessor. The partial unique index permits several cancelled successor attempts and at most one non-cancelled successor.
- Audit mapping: each event references one reservation. staff_id is populated only for a staff event; public-booking and other non-staff events have NULL. This is an event relationship, not a staff_id attribute in Reservation.
- Participation over time: the occupant list may be empty when booking. Equality between its count and guest_count is checked at check-in. One reservation segment allocates one physical room; multi-room creation produces separate reservation rows.

3.3 Functional Dependencies

Table 2 analyses each relation separately. R means only the relation named in that row, so R − {key} denotes its remaining attributes. Candidate keys are based on the declared identifiers and domain assumptions.

Table 2. Candidate keys and FDs by relation.

Relation	Candidate key(s)	FDs within this relation
Guest	guest_id	guest_id → name, email, phone
RoomType	room_type_id; type_name	room_type_id → type_name, max_guests, base_nightly_rate type_name → room_type_id, max_guests, base_nightly_rate
Room	room_id; room_number	room_id → room_number, room_type_id, is_bookable, housekeeping_status room_number → room_id, room_type_id, is_bookable, housekeeping_status
Reservation	reservation_id	reservation_id → guest_id, room_id, check_in_date, check_out_date, guest_count, booked_nightly_rate, created_at, status, actual_check_in_at, actual_check_out_at, previous_reservation_id
ReservationGuest	reservation_guest_id	reservation_guest_id → reservation_id, full_name
Staff	staff_id; username	staff_id → R − {staff_id} username → R − {username}
ReservationAudit	audit_id	audit_id → reservation_id, staff_id, source, action, occurred_at, reason, before_data, after_data

- The non-null continuation subset additionally has the intended dependency previous_reservation_id → guest_id, check_in_date. This is conditional on being a continuation and is analysed under Reservation in Section 3.4.4; it is not an unconditional FD for roots with NULL predecessors.
- Different orders for one room can have different agreed prices, so room_id does not determine booked_nightly_rate. A foreign key also does not, by itself, make its column a candidate key in the referencing relation.

3.4 Normalisation Analysis

The seven numbered analyses below follow the relation order in Table 2. Normal-form conclusions use the stated FDs; cross-row business constraints are addressed separately in Section 4.3.

3.4.1 Guest

- guest_id is the only stated candidate key and determines name, email and phone. Each row represents one booking contact; name and phone are not assumed unique.
- Under these FDs, every non-trivial determinant is a superkey, so Guest satisfies BCNF and therefore 3NF. No further decomposition is needed.

3.4.2 RoomType

- room_type_id and the non-null unique type_name are alternate candidate keys. Each determines the other key, max_guests and base_nightly_rate.
- Both determinants are superkeys, so RoomType satisfies BCNF under the stated FDs. Separating it from Room prevents a room type's capacity and current base price being repeated for every physical room.

3.4.3 Room

- room_id and the non-null unique room_number each determine the remaining Room attributes. room_type_id is a foreign key, not a candidate key of Room.
- The stated determinants are superkeys, so Room satisfies BCNF. A hypothetical combined Room/RoomType table would have room_id → room_type_id → type attributes; the implemented split is lossless because the shared room_type_id determines RoomType.

3.4.4 Reservation

- For the basic reservation attributes, reservation_id determines the remaining attributes; no room/contact identifier determines an entire order. Under those base FDs, the relation without its continuation link has the BCNF structure.
- For non-null predecessor rows, the intended previous_reservation_id → guest_id, check_in_date dependency has a non-superkey determinant: several cancelled attempts may share a predecessor. Its RHS attributes are non-prime, so this continuation subrelation does not satisfy ordinary 3NF or BCNF. NULL-root rows require separate treatment; this is not a blanket classification of the entire SQL table.
- The current implementation retains this conditional redundancy and validates continuity in the service. A separate Continuation(next_reservation_id PK/FK, previous_reservation_id FK) is a possible refinement, not an implemented table; contact/date consistency would still need cross-table validation.

3.4.5 ReservationGuest

- reservation_guest_id determines reservation_id and full_name. A name is not a key, and one reservation can have several occupant registrations.
- Each occupant has a separate row rather than occupant1/occupant2 columns or a comma-separated list. full_name is treated as one display value. Under the stated FDs, the only determinant is a superkey, giving BCNF.

3.4.6 Staff

- staff_id and non-null unique username are candidate keys and determine the other Staff attributes. A role does not identify an employee or determine their display name, password or account state.
- Under these FDs, Staff satisfies BCNF. Employee credentials and permission state remain separate from unverified Guest contact details.

3.4.7 ReservationAudit

- audit_id identifies an event; one reservation or employee can have many events. The source/staff-presence rule is a conditional nullability constraint, not an FD that identifies which employee acted.
- before_data and after_data deliberately store historical JSONB payloads. Their internal structure is not given a classical atomic-attribute decomposition here; an unconditional BCNF claim for the whole audit design is therefore not established. The snapshots are historical evidence, not a second source of current master data.

3.5 Final Design Decisions and Trade-offs

- Agreed price: booked_nightly_rate records the price confirmed for that segment, including a manager-approved fixed discount. It is distinct from the room type's current base price; later base-price changes do not rewrite old agreements.
- Continuation history: self-reference connects separately priced segments and retains earlier room/date history. This keeps the workflow simple but retains the conditional dependency discussed in Section 3.4.4.
- Contact versus occupants: Guest stores the person making the booking; ReservationGuest records who stays. guest_count is a planned quantity, so it may differ from the initially incomplete roster; the roster is validated at check-in.
- Historical snapshots: audit events retain the old and new arrangement when an eligible pre-arrival amendment updates the same reservation ID. Current master values remain in their own business tables.
- Derived information: the amount is computed from planned nights × agreed nightly rate. Reservation does not duplicate contact names, room numbers or room-type names. The current model has neither daily-price records nor a persistent group-order relation.

4. SQL, Integrity and Transactions

4.1 PostgreSQL Implementation

- Database environment: PostgreSQL 16.14 is used with Australia/Sydney hotel time. comp9120_hotel contains the demonstration data; comp9120_hotel_test is reserved for checks. The installed schema contains seven tables, 48 attributes and seven FKs.
- Data: schema.sql and seed.sql create the minimal fixture: 4 contacts, 3 room types, 6 rooms, 14 reservations and 11 occupant registrations. seed_more.sql expands this to 50, 8, 40, 200 and 351 respectively. These are reproducible seed counts rather than live counts after user operations.
- Initialisation: import schema.sql → seed.sql → seed_more.sql → staff.sql → reservation_audit.sql into a new database. For an existing database, apply additive migrations rather than rerunning the schema or seed. Staff credentials are generated locally, not stored in SQL seed files.
- Database interface: the application retains 28 positional SQL templates in tianyishen/queries.py; newer statements are also present in service.py. The separate database/queries.py has 25 named templates for standalone verification. These are two interfaces, not an assertion that the application has only 28 statements in total.
    - Python application: This application uses psycopg2-binary 2.9.13 to connect Python to PostgreSQL. Flask 3.1.3 provides the web interface; db.py is responsible for connecting to the database; web.py handles web requests; and service.py checks business rules and executes database operations.
    - Connection configuration: `db.py` calls `psycopg2.connect()` using `PGHOST`, `PGPORT`, `PGDATABASE` and `PGUSER`. `HOTEL_SCHEMA` specifies the schema. The password is read from `PGPASSWORD` or from a local password file specified by `PGPASSFILE`. These settings must match your local installation environment. The connection uses `READ COMMITTED` mode, supports manual commits and rollbacks, and uses Sydney time.
    - Running the application: Run ‘setup_web.py’ using a database administrator account. This will create a restricted ‘hotel_app’ role and configure the local connection and session settings. Subsequently, ‘run_web.py’ will start the application at http://127.0.0.1:8080 using the `hotel_app` credentials.
    - Start-up command line:

      python -m pip install -r tianyishen/requirements.txt

      python tianyishen/setup_web.py

      python tianyishen/run_web.py

4.2 Representative SQL Operations

To reproduce the four groups below, initialise a fresh test schema with schema.sql and seed.sql only; omit seed_more.sql. The 14-order outputs were reproduced on 2026-10-25 in a disposable test schema. %s denotes a bound parameter. Each group lists purpose, SQL, parameters, actual output and interpretation.

4.2.1 Availability — joins and NOT EXISTS

1. Purpose: find an open room with sufficient capacity whose planned interval is not blocked by a confirmed or checked-in reservation.

```
SELECT rm.room_id, rm.room_number, rt.type_name,
       rt.max_guests, rt.base_nightly_rate,
       (%s::date-%s::date)*rt.base_nightly_rate AS total_quote
FROM room rm JOIN room_type rt USING(room_type_id)
WHERE rm.is_bookable AND rt.max_guests >= %s
  AND (%s::integer IS NULL OR rm.room_type_id=%s)
  AND NOT EXISTS (
    SELECT 1 FROM reservation r WHERE r.room_id=rm.room_id
      AND r.status IN ('confirmed','checked_in')
      AND r.check_in_date < %s AND r.check_out_date > %s)
ORDER BY rm.room_id;
```

2. Parameters, in order: departure 2026-10-28, arrival 2026-10-27, guests 1, room-type filter NULL, repeated filter NULL, departure 2026-10-28, arrival 2026-10-27.

```
room_id | room_number | type_name | capacity | nightly | total
3       | 201         | Double    | 2        | 180.00  | 180.00
4       | 202         | Double    | 2        | 180.00  | 180.00
```

3. Interpretation: both Double rooms are available. The strict overlap comparisons allow adjacent stays; cancelled history does not block a room. A current dirty state alone does not rule out all future bookings.

4.2.2 Create a reservation — INSERT … RETURNING

1. Purpose: store one priced room allocation and return its generated ID, agreed nightly price and derived stay amount.

```
BEGIN;
INSERT INTO reservation
 (guest_id,room_id,check_in_date,check_out_date,guest_count,
  booked_nightly_rate,previous_reservation_id)
VALUES (%s,%s,%s,%s,%s,%s,%s)
RETURNING reservation_id,booked_nightly_rate,
 (check_out_date-check_in_date)*booked_nightly_rate AS total;
ROLLBACK;
```

2. Parameters: contact 1, room_id 2, arrival 2026-11-14, departure 2026-11-16, guests 1, agreed nightly price 120.00, predecessor NULL.

```
reservation_id | booked_nightly_rate | total
1015           | 120.00              | 240.00
```

3. Interpretation: two nights cost 240.00 in currency-unspecified demonstration units. This isolated SQL example was rolled back; the fixture still had 14 orders afterwards. The generated ID is not promised on every later run. Application validation and the complete transaction are explained by the Python author in Sections 4.4–4.5.

4.2.3 Statistics — LEFT JOIN and conditional aggregation

1. Purpose: count bookings and cancellations by room type and calculate planned non-cancelled booking value, retaining types with no bookings.

```
SELECT rt.type_name, COUNT(r.reservation_id) AS bookings,
 COUNT(*) FILTER (WHERE r.status='cancelled') AS cancelled,
 COALESCE(SUM(CASE WHEN r.status<>'cancelled'
   THEN (r.check_out_date-r.check_in_date)
        *r.booked_nightly_rate ELSE 0 END),0) AS value
FROM room_type rt LEFT JOIN room rm USING(room_type_id)
LEFT JOIN reservation r ON r.room_id=rm.room_id
GROUP BY rt.room_type_id,rt.type_name
ORDER BY rt.room_type_id;
```

2. Parameters: none. Actual output from the unchanged minimal fixture:

```
type_name | bookings | cancelled | value
Single    | 6        | 0         | 1300.00
Double    | 4        | 1         | 1080.00
Family    | 4        | 2         |  940.00
```

3. Interpretation: this is planned booking value, not collected revenue. Occupant rows are not joined, so they do not multiply an order's value. These are minimal-fixture results, not current expanded totals.

4.2.4 Contact and occupants — related facts in separate tables

1. Purpose: show that the booking contact can differ from the people actually staying.

```
SELECT g.name AS contact, rg.full_name AS occupant
FROM reservation r JOIN guest g USING(guest_id)
JOIN reservation_guest rg USING(reservation_id)
WHERE r.reservation_id=%s
ORDER BY rg.reservation_guest_id;
```

2. Parameter: reservation_id 1001. Actual output:

```
contact    | occupant
Alice Chen | Bob Chen
Alice Chen | Carol Chen
```

3. Interpretation: one contact can arrange accommodation for several other people. The join retrieves names without duplicating them in Reservation.

4.3 Integrity Constraints

1. Keys and references: identity PKs identify rows; room_number, type_name and username are non-null unique keys. Seven foreign keys reject missing parents and use ON DELETE RESTRICT to preserve referenced history.
2. Domains: CHECK/NOT NULL rules reject whitespace-only required names/phones, nonpositive party sizes/capacities, negative or NaN prices, invalid status values, nonfinite dates/timestamps, zero/negative planned stays and self-predecessors. PostgreSQL CHECK constraints do not provide general cross-table guarantees [1].
3. Actual-time consistency: confirmed/cancelled rows have neither actual timestamp; checked_in has arrival only; checked_out has ordered actual arrival and departure. These row-level checks are separate from allowed service workflow transitions.

4. Conditional uniqueness: the following indexes enforce at most one actually checked-in order per room and one non-cancelled successor per predecessor. They do not prevent every planned interval overlap.

```
CREATE UNIQUE INDEX uq_room_checked_in
 ON reservation(room_id) WHERE status='checked_in';
CREATE UNIQUE INDEX uq_reservation_successor
 ON reservation(previous_reservation_id)
 WHERE previous_reservation_id IS NOT NULL
   AND status<>'cancelled';
```

5. Business-rule boundary: capacity, roster completeness, date conflicts, housekeeping transitions, quote confirmation and continuation consistency require service validation. The ordinary active-date index is a query access path, not an exclusion constraint. The lock/transaction explanation belongs in Sections 4.4–4.5.
6. Audit integrity: each event references an order; staff events require staff_id, while other sources require NULL. The application role can SELECT/INSERT audit events but cannot UPDATE/DELETE them. Owner SQL remains outside the controlled service audit boundary.

Table 3 summarises rejected database operations from the constraint evidence; examples are executed against test fixtures, not the demonstration database.

Table 3. Rejected operations and database responses.

Invalid operation	Expected refusal	Evidence interpretation
Insert an order with a missing contact/room reference	23503 — FK violation	No parent reference can be invented.
Insert a required NULL value	23502 — NOT NULL violation	Required attributes remain present.
Insert a duplicate room number or a second checked-in order for one room	23505 — unique violation	Ordinary or partial uniqueness is enforced.
Insert a zero-night stay, invalid status or inconsistent actual timestamps	23514 — CHECK violation	The row violates a declared domain/time rule.
Delete a parent that is still referenced	23503 — restricted deletion	Referenced history is preserved.

4.4 Transaction Scenario

activate_continuation() handles a room change in one transaction. The continuation is created separately beforehand. After validation and locking, one connection checks out the old reservation, marks the old room dirty, checks in the next reservation and writes two audit records. Both actual timestamps use the same event time. conn.commit() saves all changes together; an error before commit triggers conn.rollback(), preventing a partially completed room change.

```
    event = _event_time(cur)
    cur.execute(queries.CHECK_OUT, (event, previous_id))
    if previous['room_id'] != next_stay['room_id']:
        cur.execute(queries.SET_HOUSEKEEPING, ('dirty', previous['room_id']))
    cur.execute(queries.CHECK_IN, (event, next_reservation_id))
    _history(cur, previous_id, 'activate_continuation', before_previous)
    _history(cur, next_reservation_id, 'activate_continuation', before_next)
    conn.commit()
except Exception:
    conn.rollback()
    raise
finally:
    cur.close()
    conn.close()
```

4.5 Failure and Concurrency Reasoning

If a step fails before commit,conn.rollback() undoes that transaction's changes. In our SQL demo, we cause a divide-by-zero error and run ROLLBACK. The bookings and rooms return to their earlier states.

Under READ COMMITTED, the first request locks the room while creating a booking. A second request for the same room waits. After the first booking is saved, the second checks the dates again and rejects any overlap. All booking writes must follow these steps; a normal date index alone cannot prevent double bookings.

```
try:
    room = _room(cur, room_id, lock=True)
    validate_stay(check_in, check_out, guests, _today(cur))
    if not room['is_bookable']:
        raise BookingError('Room is closed for booking.')
    _check_capacity(room, guests)
    cur.execute(queries.GET_GUEST, (guest_id,))
    if cur.fetchone() is None:
        raise BookingError('Booking contact not found. Register the contact first.')
    # Recheck availability after locking.
    _check_availability(cur, room_id, check_in, check_out)
    cur.execute('SELECT base_nightly_rate FROM room_type WHERE room_type_id=%s',
                (room['room_type_id'],))
    room['base_nightly_rate'] = cur.fetchone()[0]
    rate, discount_reason = _confirmed_rate(cur, room, quoted_base, discount_rate, reason)
    cur.execute(queries.INSERT_RESERVATION, (guest_id, room_id, check_in, check_out,
                                             guests, rate, None))
    result = cur.fetchone()
    _history(cur, result[0], 'create', reason=discount_reason,
             details={'reference_rate': str(room['base_nightly_rate']), 'discounted': discount_rate is not None})
    conn.commit()
    return result
except Exception:
    conn.rollback()
    raise
```

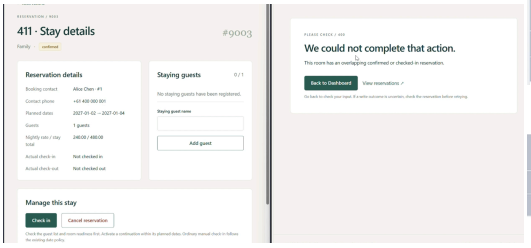
4.6 Testing and Evidence of Correctness

1. Constraint checks: database/evidence/database_validation.txt records 43 constraint/positive checks plus one explicit direct-SQL bypass boundary. These are not 43 separate application features.

```
C01 guest PK|PASS 23505
C02 name required|PASS 23502
C07 capacity positive|PASS 23514
C11 room type FK|PASS 23503
C36 one noncancelled successor|PASS 23505
```

note:Selected database constraint test results.

2. SQL examples: the same record contains 35 fixed-input cases for the standalone 25-template interface. The four report examples were additionally reproduced in a fresh minimal-fixture test schema and the insertion rolled back.
3. Expanded data: data_expansion_validation.txt and data_expansion.json record original-row preservation, counts, occupancy/date consistency and repeat-import rollback. Evidence is local; a second-computer or production-load test is not claimed.
4. 4. Python/service checks: Screenshots show continuation activation, rollback and rejection of a conflicting booking.



	reservation_id integer	room_number text	status text	housekeeping_status text	actual_check_in_at timestamp with time zone	actual_check_out_at timestamp with time zone
1	9001	102	checked_...	clean	2026-10-24 15:00:00+11	[null]
2	9002	202	confirmed	clean	[null]	[null]

	reservation_id integer	room_number text	status text	housekeeping_status text	actual_check_in_at timestamp with time zone	actual_check_out_at timestamp with time zone
1	9001	102	checked_o...	dirty	2026-10-24 15:00:00+11	2026-10-25 14:45:12.416005+...
2	9002	202	checked_in	clean	2026-10-25 14:45:12.416005+...	[null]

References

[1] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, §5.4 Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html (accessed 25 October 2026).

AI Use Acknowledgement

Member1: Jinxing Wu
I used OpenAI GPT-6.1-sol through Codex with High reasoning effort for the following purposes:

- Project selection: AI helped us explore and select the hotel reservation database project.
- Database implementation: After we completed the ER diagram, RM diagram and overall constraint design, AI assisted in refining the business rules, writing SQL and setting up the PostgreSQL database.
- Sample data: The sample data used for development, testing and demonstration were generated by AI.
- Demonstration website: The web interface was generated by AI primarily to demonstrate the database operations.
- Report preparation: AI assisted with organising and editing the report, and generated the report's tables and visual summaries.

Member2: Tianyi Shen

AI usage declare:

1. Python code: The code demonstrating how to reference Sydney time was generated using Codex, and the relevant rules were refined in `service.py`.
2. Video production: The video cover and closing sequence were generated by AI based on the project theme; whilst AI assisted with scheduling shot timings, I was personally responsible for video recording and editing.
3. Report Content: I used AI to identify the corresponding rule-definition code and standardised the terminology and phrasing.
4. Checking and Debugging: I used AI to assist with interpreting error messages, checking the code, organising test steps, and verifying the programme and demonstration results.

Member 3: Zirui Meng

I declare that I used AI (gemini, chatgpt) for:

1. Business rules design and ER diagram. I used AI to generate business rules and the initial version of the ER diagram for our project.
2. Creating the poster: I used AI to generate images for the poster.
3. Report Contents: I used AI to summarize the main contents and wrote them in the report.

The group remains responsible for the submitted design, implementation and report.